1. 事件机制

事件机制是PyQt5核心概念之一，掌握事件机制对于开发GUI应用程序至关重要。在PyQt5中，事件机制用于处理应用程序与用户之间的交互。其中，事件可以是用户行为，如按键、鼠标点击、移动窗口等。也可以是系统通知，如定时器触发、网络消息到达等。

思考：

1. Qt应用程序中的事件的来源？

Qt中的事件来源于系统消息，经Qt平台(QApplication)转换封装为Qt事件。

Qt事件是一个QEvent对象。

系统消息主要是操作系统捕捉到的底层软、硬件信息，比如，触屏时触发中断，操作系统将这个触屏的信息封装成系统消息发送到更上层的平台

QT的所有GUI程序都是事件驱动的。

2. Qt中的信号的来源？

我们既然知道了Qt事件的来源，是不是也能认为信号也来自系统消息的转换？答案是否定的。

那么Qt信号的产生不来自系统，就只能来源于Qt内部？

要搞清这个问题，我们首先要知道Qt事件处理机制。

3. QT事件处理机制：

事件主要由用户触发（用户与界面交互），但也可能有其他触发方式：例如网络连接、window manager或定时器。

```
例如：  用户移动鼠标，会产生鼠标移动事件
        用户点击鼠标，会产生鼠标点击事件
        用户滚动鼠标滑轮，会产生滑轮事件
        用户键盘输入，会产生键盘输入事件
```

当我们调用QApplication的exec_()方法时会使程序进入主事件循环；主循环会获取并分发事件， 执行对应的事件处理函数。

在事件模型中，有三个参与者：

- 事件源
- 事件对象
- 事件接收者

其中：

- 事件源：是状态发生变化的对象
- 事件(对象)封装了事件源中状态的变动。
- 事件接收者是要通知的对象， 事件源对象将事件处理的工作交给事件接收者。

事件处理机制：

作者远见元宇宙实验中心
yyzlab.com.cn

Qt事件(QEvent对象)产生之后，会被分发到QWidget对象（QWidget是所有与用户交互类的父类），在QWidget的event(QEvent*) 函数中进行事件处理，该函数根据事件类型调用不同的事件处理函数。

例如：

```
用户操作鼠标去点击按钮， 有事件对象产生， 按钮对象中的 event() 事件分发函数被触发，这个函数会根据
事件的类型来判断，如果是鼠标点击事件，就会调用按钮中的鼠标点击事件处理函数！
```

但是我们之前一直用的是 按钮点击 产生clicked信号了，信号与槽函数关联就可以执行按钮业务！ 现在又说按钮点击，是产生了
事件，并会执行事件处理函数！

信号与事件有什么关系吗?

通过例子验证发现， 按钮的clicked信号是在鼠标点击事件处理中被触发的！

继承QPushButton类，重写鼠标点击事件处理函数，点击按钮，发现鼠标点击事件处理函数被执行了，而与按钮clicked信号

关联的槽函数没有被执行：

```
分析原因： 是否存在信号、 是否与槽函数关联、信号是否被触发
最终得到： 信号没有被触发
验证：     在按钮类的鼠标点击事件处理函数中， clicked.emit()  槽函数正常执行！
```

故： 信号是在某一特定事件发生时被触发的！

事件-> 对应控件的事件处理函数->触发信号->信号与槽函数关联->执行对应的槽函数

1.1 鼠标事件

1.1.1 鼠标点击事件

mousePressEvent()

案例： 鼠标在哪点击，标签在哪显示

思考：

鼠标上的按键有左键、右键、中间按键，每一个按键在实际中的应用都是不一样的！

如何知道鼠标点击的是哪一个按键呢?

根据按钮来判断：QMouseEvent.button()-> Qt.MouseButton 枚举

1.1.2 鼠标移动事件

mouseMoveEvent()

注意：

```
要想让鼠标移动事件被处理，就要打开鼠标追踪，不然只有按下鼠标左键并移动时才会产生鼠标移动事件，并执行对应的事件处理函数！
setMouseTracking(True)
```

案例： 鼠标移动到哪里， 图片在哪里显示

华清远见元宇宙实验中心 yyzlab.com.cn

1.2 键盘输入事件

keyPressEvent()： 当键盘有输入的时候，产生事件对象， 界面对象的键盘输入事件被执行

案例：在界面上做键盘输入，验证输入的是什么内容

```
def keyPressEvent(self, a0):
    """
    键盘输入事件处理函数
    :param a0:
    :return:
    """
    key = a0.key()
    print("key = {}".format(key))  # 字符串的拼接

    text = a0.text()
    print("text = {}".format(text))
```

PressWidget › keyPressEvent()

Run　demo_020

```
D:\soft\miniconda\envs\python_38\python.exe E:\AI_git\07-案例源码\PyQT\001\
key = 16777235
text =
key = 81
text = q
key = 65
text = a
key = 69
text = e
key = 49
text = 1
key = 50
text = 2
key = 51
```

华清远见元宇宙实验中心 yyzlab.com.cn

案例：

编辑框限制输入

```
《1》编辑框只能输入数字
《2》编辑框啥都不能输入
```

实现：

```
《1》编辑框只能输入数字
```

重定义一个编辑框子类，继承QLineEdit
重写 keyPressEvent()事件函数
在这个函数内对键盘输入的值进行判断，符合要求的显示

```
class myLineEdit(QLineEdit):
    def __init__(self, parent):
        super(myLineEdit, self).__init__(parent)
        self.setParent(parent)

    # 重写键盘输入事件处理函数
    1 usage
    def keyPressEvent(self, a0):
        # 判断键盘输入的是什么
        data = a0.text()
        if '0' <= data <= '9':  # 只能输入数字
            self.setText(self.text() + data)
```

《2》编辑框啥都不能输入

可以按下述的方法去实现，函数体内什么都不用做

```
3 usages
class myLineEdit(QLineEdit):
    def __init__(self, parent):
        super(myLineEdit, self).__init__(parent)
        self.setParent(parent)

    # 重写键盘输入事件处理函数
    1 usage
    def keyPressEvent(self, a0):
        pass
        # # 判断键盘输入的是什么
        # data = a0.text()
        # if '0' <= data <= '9':  # 只能输入数字
        #     self.setText(self.text() + data)
```

可以重写 event()事件分发函数， 判断事件类型是不是键盘输入

```
如果是，直接return True
否则，执行父类的event
```

华清远见宇宙实验中心 yyzlab.com.cn

```python
    # 重写事件分发函数
    def event(self, a0):  # 返回值是 bool类型
        """
        在执行keyPressEvent()函数之前，先执行的event事件分发函数，
        在这函数中根据事件的类型--type()判断是什么事件，才执行对应的事件处理函数
        :return:
        """
        if a0.type() == QEvent.KeyPress:
            print("事件类型是键盘输入事件")
            # 执行键盘输入事件处理函数
            # self.keyPressEvent(QKeyEvent(a0))
            return True
        else:
            return super(myLineEdit, self).event(a0)
```

按父类中的方案执行

思考：

> 那一个界面中的多个组件都要对某一个事件进行额外的处理，如果都采用上述方式去处理，工作量会变得很大，类也会很多，使用起来就不方便了，如果能用一个方法搞定，效果会更好！

实现：

> 在界面类中重写事件过滤 eventFilter()
> 注意： 对哪个控件做处理，就需要给哪个控件安装事件过滤器

```python
4 usages
class KeyPressWidget(QWidget):
    def __init__(self):
        super(KeyPressWidget, self).__init__()
        print(self)
        self.setWindowTitle("编辑框的键盘输入事件")

        self.edit = myLineEdit(self)
        self.edit.move(100, 100)
        # # 安装事件过滤
        self.edit.installEventFilter(self)

    # 重写 事件过滤函数
    # 参数1： 对象    参数2：事件
    def eventFilter(self, a0, a1):
        # 如果对象是 编辑框对象
        if a0 == self.edit:
            # 如果事件类型是 键盘输入
            if a1.type() == QEvent.KeyPress:
                return True
            else:
                # 按照父类中的操作去执行
                return super(KeyPressWidget, self).eventFilter(a0, a1)
        else:
            return super(KeyPressWidget, self).eventFilter(a0, a1)
```

华清远见元宇宙实验中心 yyzlab.com.cn

1.3 绘图事件

paintEvent()

之间遗留下来的问题： 使用QPalette给界面设置背景图片，不能实现背景图自适应。也就是窗口变大变小，背景图片并不能跟着发生变化

解决方案：

```
重写绘图事件
```

什么时候执行绘图事件呢？

1. 当窗口部件第一次显示时，系统会自动产生一个绘图事件。
2. repaint()与update()函数被调用时。

   ```
   区别中关键点是：
         repaint()是立即调用paintEvent()，而update()是几次执行才调用一次paintEvent()。
         这样update()会造成这样的结果：
                  paintEvent()中的任务没有执行完，就又被update()。
                  paintEvent()中被积压的任务越来越多。
   ```

   在绝大多数情况下，update()更好，因为它允许Qt来优化速度并且防止闪烁。
3. 当窗口部件被其他部件遮挡，然后又再次显示出来时，就会对隐藏的区域产生一个重绘事件。
4. 重新调整窗口大小时

实现：

设置背景图片

绘制一个圆

绘制一条线

综合应用：

```
1.鼠标移动到哪，圆跟着到哪
2.键盘输入上下左右按键，  圆跟着上下左右移动
3.鼠标滚动滑轮，圆放大或者缩小
```

分析：

```
1和2其实本质就是更新  圆外接矩形左上角点的坐标值    即  x和y的值
3本质上就是更新  圆的半径
```

1.4 关闭事件

closeEvent()

在实际中，当我们要退出程序的时候，一般都会询问： 确定要退出吗？

而我们现在的程序退出，直接要退出了！ 那如果是误操作，实际并不想退出呢？

解决方案：

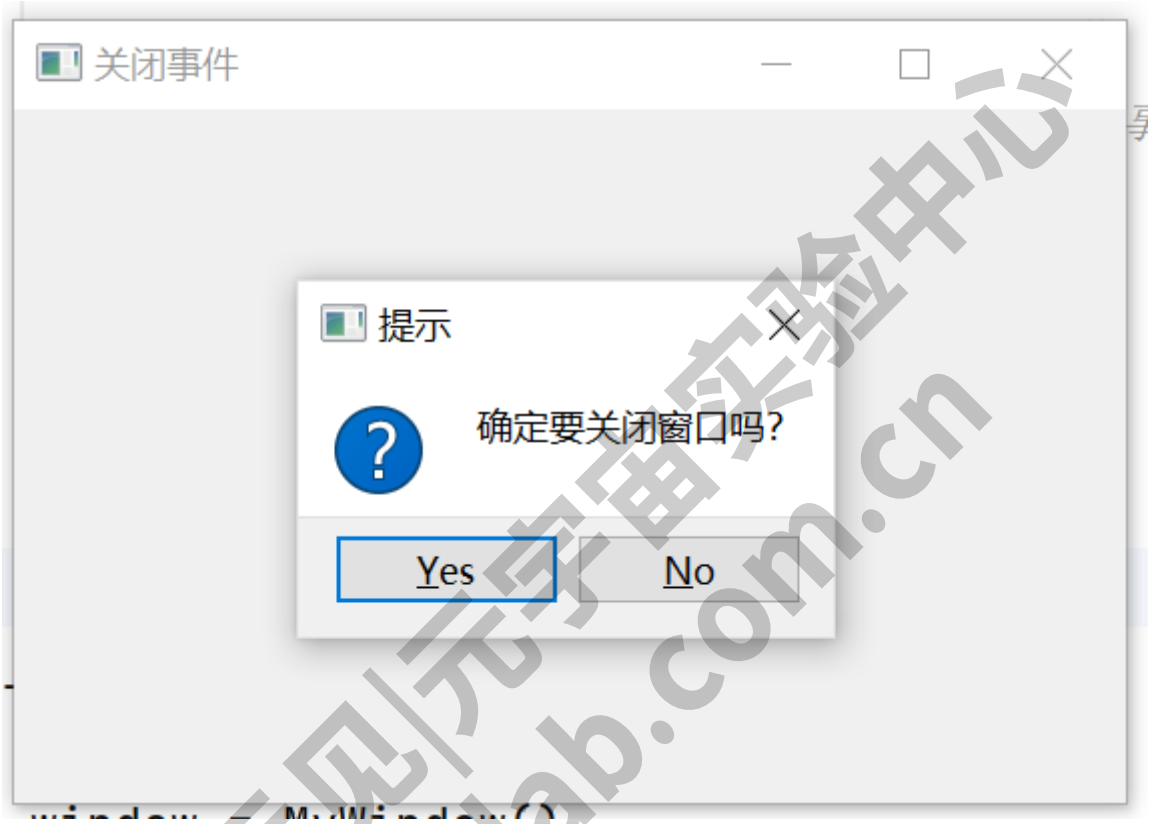
华清远见元宇宙实验中心 yyzlab.com.cn

```
就是重写closeEvent()事件处理函数
```

何时触发closeEvent()事件：

```
用于处理窗口关闭事件。当用户点击窗口右上角的关闭按钮或调用窗口的close()函数时，closeEvent方法会被触发。
closeEvent中 默认接收并关闭窗口
```

实现：



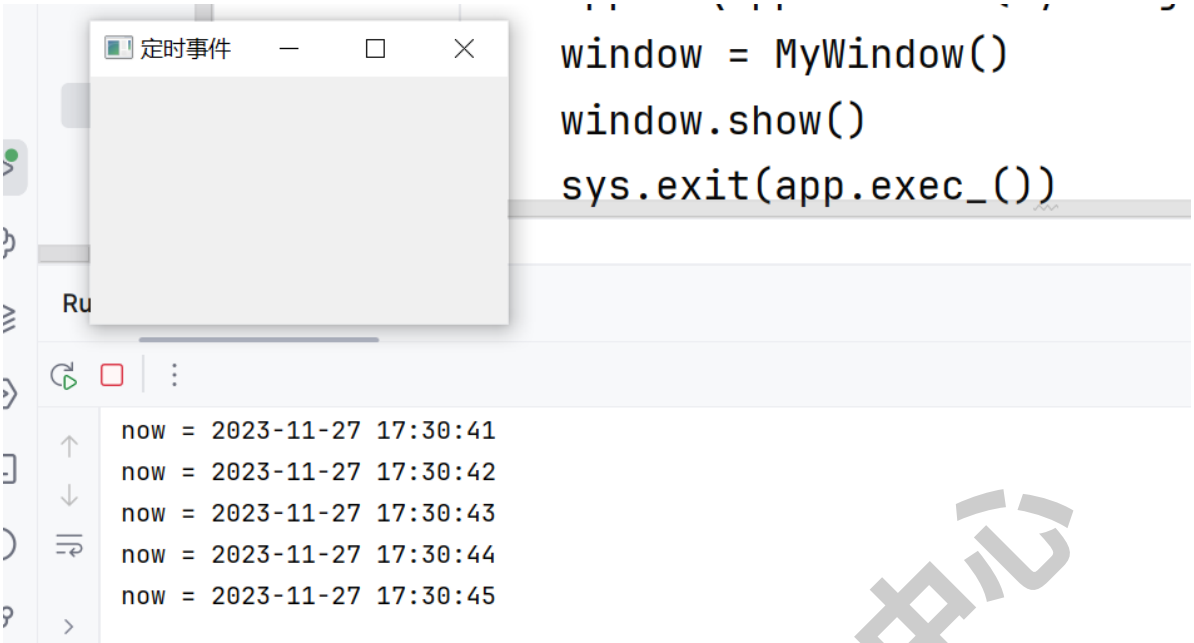
1.5 定时器事件

timerEvent()

启动定时器之后，每当规定的时间一到，就会触发定时器事件处理函数

案例： 每一秒打印一下系统时间

华清远见元宇宙实验中心 yyzlab.com.cn



华清远见元宇宙实验中心
yyzlab.com.cn